

Binary Information

Binary Name => good kitty

Language => C/C++

Arch => x86x64

Platform => Unix/Linux

```
$ file crack
```

crack: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,

interpreter /lib64/ld-linux-x86-64.so.2,

BuildID[sha1]=8c584d707909182cb49dab6ebe51cca2217ab1ed, for GNU/Linux 3.2.0, not stripped

Analysis

Static Analysis

Below is the disassembly of the **main()**. I have only pasted the essential part

```
..... (SNIP) .....  
0x00005555555554f0 <+580>: test  al,al  
0x00005555555554f2 <+582>: je   0x5555555555562 <main+694>  
0x00005555555554f4 <+584>: lea  rdi,[rip+0xb14]    # 0x55555555600f  
0x00005555555554fb <+591>: call 0x5555555555090 <puts@plt>  
0x0000555555555500 <+596>: mov  rax,QWORD PTR [rsp+0xa8]  
0x0000555555555508 <+604>: sub  rax,QWORD PTR fs:0x28  
0x0000555555555511 <+613>: jne  0x5555555555570 <main+708>  
0x0000555555555513 <+615>: mov  eax,0x0  
0x0000555555555518 <+620>: add  rsp,0xb8  
0x000055555555551f <+627>: pop  rbx  
0x0000555555555520 <+628>: pop  rbp  
0x0000555555555521 <+629>: ret  
0x0000555555555522 <+630>: mov  edx,DWORD PTR [rsp+0xc]  
0x0000555555555526 <+634>: add  edx,0x1  
0x0000555555555529 <+637>: mov  DWORD PTR [rsp+0xc],edx  
0x000055555555552d <+641>: mov  edx,DWORD PTR [rsp+0xc]  
0x0000555555555531 <+645>: movsxd rdx,edx  
0x0000555555555534 <+648>: cmp  rdx,rax  
0x0000555555555537 <+651>: jge  0x555555555554d9 <main+557>  
0x0000555555555539 <+653>: mov  edx,DWORD PTR [rsp+0xc]  
0x000055555555553d <+657>: cmp  edx,0x7  
0x0000555555555540 <+660>: ja   0x555555555554d9 <main+557>  
0x0000555555555542 <+662>: mov  ecx,DWORD PTR [rsp+0xc]  
0x0000555555555546 <+666>: mov  edx,DWORD PTR [rsp+0xc]  
0x000055555555554a <+670>: movsxd rcx,ecx
```

```

0x000055555555554d <+673>: movsxd rdx,edx
0x0000555555555550 <+676>: movzx ebx,BYTE PTR [rsp+rdx*1+0x10]
0x0000555555555555 <+681>: cmp  BYTE PTR [rsp+rcx*1+0x60],bl
0x0000555555555559 <+685>: je   0x5555555555522 <main+630>
0x000055555555555b <+687>: mov  BYTE PTR [rsp+0xb],0x0
0x0000555555555560 <+692>: jmp  0x5555555555522 <main+630>
0x0000555555555562 <+694>: lea  rdi,[rip+0xa9b]    # 0x5555555556004
0x0000555555555569 <+701>: call 0x5555555555090 <puts@plt>
0x000055555555556e <+706>: jmp  0x5555555555500 <main+596>
0x0000555555555570 <+708>: call 0x55555555550b0 <__stack_chk_fail@plt>

```

End of assembler dump.

gef> x/s 0x555555556004

0x555555556004: "bad kitty!"

gef> x/s 0x55555555600f

0x55555555600f: "good kitty!"

gef>

Program Workflow

- Takes a password as user input.
- If password is correct **good kitty!** is printed otherwise **bad kitty!** is printed.

Dynamic Analysis

- I setup the breakpoints at **main()** and at 0x00005555555554f0 <+580>: test al,al.

gef> i b

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep y		0x00005555555552ac <main>	
					breakpoint already hit 1 time
2	breakpoint	keep y		0x00005555555554f0 <main+580>	

- I ran the program after that, and I noticed somewhere between breakpoint 3 and breakpoint 4 there was a **unique string** which got placed inside the stack. Below is the image of the stack condition when that happens.

```

[ Legend: Modified register | Code | Heap | Stack | String ]

Registers:
$rax : 0x0
$rbx : 0x34
$rcx : 0x5
$rdx : 0x0
$rsp : 0x00007fffffffdb00 + 0x000000000300000
$rbp : 0x00007fffffffdbf8 + "passwordenter the right password" ← user input string
$rc1 : 0x00007fffffffdb00 + "sourceVU"
$rdi : 0x0
$rip : 0x0000555555554f0 + <main+0244> test al, al
$ir : 0x0
$ir9 : 0x0
$ir10 : 0x0
$ir11 : 0x202
$ir12 : 0x0
$ir13 : 0x00007fffffffdb00 + 0x00007fffffff21d + "COLORFGG=15;0"
$ir14 : 0x00007fffffffdb00 + 0x00007fffffff2310 + 0x0000555555554000 + 0x0001010246ac457f
$ir15 : 0x0000555555557d90 + 0x000055555555100 + <-0x_global_dtors_aux+0000> <under64>
$eflags: (ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume virtualx86 identification)
cs: 0x23  eip: 0x20  iopl: 0x00  iopl: 0x00  iopl: 0x00  iopl: 0x00

Stack:
0x00007fffffffdb00 +0x0000: 0x000000000300000 + $rsp
0x00007fffffffdb00 +0x0000: 0x0000000000000000
0x00007fffffffdbf8 +0x0010: "00sG04M0passwordenter the right password"
0x00007fffffffdbf8 +0x0018: "passwordenter the right password" + $rbp
0x00007fffffffdbf8 +0x0020: enter the right password"
0x00007fffffffdbf8 +0x0028: "e right password"
0x00007fffffffdbf8 +0x0030: "password"
0x00007fffffffdbf8 +0x0038: 0x0000000000000000

Code: x86:164
0x555555554e5 <main+0239> and eax, edx
0x555555554e7 <main+023b> mov BYTE PTR [esp+0b], al
0x555555554e9 <main+023f> movzx eax, BYTE PTR [rsp+0-b]
0x555555554f0 <main+0244> test al, al
0x555555554f2 <main+0246> je 0x555555555002 <main+094>
0x555555554f4 <main+0248> lea rdi, [rip+0-b14] # 0x55555555500f
0x555555554f6 <main+024f> call 0x555555555000 <puts@plt>
0x55555555500 <main+0254> mov rax, QWORD PTR [rsp+0xa8]
0x55555555500 <main+025c> sub rax, QWORD PTR fs:[0-28]

Threads:
[0] Id 1, Name: "crack", stopped 0x555555554f0 in main (), reason: BREAKPOINT

Trace:
[0] 0x555555554f0 → main()

```

Testing our input

```

$ ./crack
enter the right password
00sG04M0
good kitty!

```

Yeah!!! we were able to solve this crackme challenge.